

Centro educativo

Código	Centro	Concello	Ano académico
15005397	I.E.S. Fernando Wirtz Suárez	A Coruña	2025-2026

Ciclo formativo

Código da familia profesional	Familia profesional	Código do ciclo formativo	Ciclo formativo	Grao	Réxime
FP16	Informática e Comunicacóns	SIFC03	Desenvolvemento de Aplicacións Web	Superior	Dual

Módulo profesional

Código MP/UF	Nome
MP0616	Proxecto de Desenvolvemento de Aplicación Web Equivalencia en créditos ECTS: 5. Código: MP0616. Duración: 26 horas

Profesorado responsable

Titor	Prado Espiñeira, Fernando
Equipo docente	García Ulla, Constantino Gimeno Peón, María Jesús Iglesias Gómez, Marta Pascual Vázquez, Víctor Rodríguez Diéguez, Fernando

Alumna

Alumna	Sara Barreiro Riveiro
---------------	-----------------------

Datos do proxecto

Título	Explora
---------------	---------

Índice

1. OBXECTIVO	4
1.1. CONTEXTO E MOTIVACIÓN	4
1.2. OBXECTIVOS XERAIS E ESPECÍFICOS	5
2. DESCRICIÓN	7
2.1. DESCRICIÓN XERAL DO PROBLEMA	7
2.2. REQUISITOS FUNCIONAIS	8
2.3. REQUISITOS NON FUNCIONAIS	10
2.3. ANÁLISE DE USUARIA	12
2.4. CASOS DE USO	14
3. ALCANCE	19
3.1. FUNCIONALIDADES INCLUÍDAS	19
3.2. ASPECTOS TÉCNICOS INCLUÍDOS NO PROXECTO	21
3.3. LIMITACIÓNS DO ALCANCE	22
4. PLANIFICACIÓN	23
4.1. METODOLOXÍA DE DESENVOLVEMENTO	23
4.2. ORGANIZACIÓN EN SPRINTS	24
4.3. PLANIFICACIÓN DETALLADA DOS SPRINTS	24
4.4. FITOS DO PROXECTO	32
4.5. DIAGRAMA DE GANTT	32
5. MEDIOS A UTILIZAR	33
5.1. TECNOLOXÍAS	33

5.2. FERRAMENTAS	34
6. PRESUPOSTO	36
6.1. CUSTOS ESTIMADOS	36
7. EXECUCIÓN	37
7.1. DESEÑO DO SISTEMA	37
7.2. IMPLEMENTACIÓN	37
7.3. PROBAS E VALIDACIÓN	38
7.4. DESPREGAMENTO E DOCUMENTACIÓN	38
7.5. RESULTADOS E AVALIACIÓN	38
7.6. CONCLUSIÓNS E LIÑAS DE MELLORA	39
REFERENCIAS	40

1. Obxectivo

1.1. Contexto e motivación

Filosofía: privacidade, liberdade e independencia tecnolóxica

Na actualidade, as solucións web máis usadas para mapas dixitais implican unha gran dependencia de servizos centralizados e a cesión masiva de datos persoais a grandes empresas tecnolóxicas. Esta situación non só limita a soberanía das persoas usuarias sobre os seus propios datos, senón que tamén expón a información sensible a riscos de privacidade e uso indebido.

Por iso, existe a necesidade de crear alternativas baseadas en software libre, colaborativas e respectuosas coa privacidade, que lle permitan ás persoas usuarias controlar de forma íntegra os seus datos e contidos xeoespaciais. Neste sentido, OpenStreetMap constitúe un exemplo paradigmático dunha plataforma aberta e comunitaria que posibilita este tipo de desenvolvemento.

O proxecto presentado consiste nunha aplicación web de mapas colaborativos construída sobre tecnoloxías libres, cunha filosofía clara de control total da usuaria sobre a súa información persoal e os seus contidos. Permitirá a creación e xestión de mapas personalizados, públicos ou privados, con múltiples usuarias colaborando segundo un sistema xerárquico de permisos.

A aplicación adopta un enfoque de *privacy by design*, minimizando a recollida de datos persoais e aplicando mecanismos de seguridade que garantan a confidencialidade da información das usuarias.

Baleiro no mercado: a colaboración

Aínda que existen ferramentas que permiten compartir mapas ou localizacións, moitas delas presentan limitacións en aspectos como o control granular de permisos, a edición colaborativa estruturada ou a independencia respecto de plataformas propietarias.

Este proxecto busca cubrir esa fenda, facilitando a creación e xestión de mapas colaborativos con niveis de acceso e permisos diferenciados, fomentando a interacción e o traballo en equipo.

Funcionalidade *offline*

Outro valor diferencial é a integración da funcionalidade *offline*, que permitirá o acceso e edición dos mapas e puntos de interese mesmo sen conexión a internet, unha característica especialmente útil en contornas con conectividade limitada ou intermitente, como roteiros rurais, ou viaxes sen cobertura.

1.2. Obxectivos xerais e específicos

Obxectivos xerais

Desenvolver unha aplicación web de mapas colaborativos baseada en tecnoloxías libres que lles permita ás usuarias crear, xestionar e compartir mapas públicos ou privados, garantindo a privacidade da información e facilitando a colaboración entre múltiples usuarias mediante un sistema estruturado de permisos e funcionalidades accesibles.

Obxectivos específicos

Establécense os seguintes obxectivos específicos, que describen as funcionalidades e características principais que debe incorporar a aplicación desenvolvida.

1. Integrar mapas base de OpenStreetMap como referencia para a visualización e edición dos mapas colaborativos dentro da aplicación.
2. Permitir a creación, configuración e xestión de mapas personalizados, con opción de definir a súa visibilidade como pública ou privada.
3. Desenvolver un sistema de descubrimento de mapas públicos baseado na localización xeográfica, que lles permita ás persoas usuarias buscar mapas asociados a unha cidade, rexión ou lugar concreto e visualizar os mapas colaborativos que teñan esa localización como referencia.
4. Implementar un sistema de convite e colaboración entre usuarias, permitindo asignar diferentes roles dentro de cada mapa.
5. Desenvolver ferramentas colaborativas que permitan engadir, modificar e eliminar marcadores xeográficos, mantendo un historial de cambios que posibilite a consulta e restauración de versións anteriores.
6. Integrar a posibilidade de engadir notas asociadas aos marcadores, permitindo definir a súa visibilidade segundo o contexto do mapa ou da persoa usuaria.

7. Implementar un sistema de categorías para os marcadores, con atributos visuais como cor e iconas, que facilite a organización da información e o filtrado personalizado dos elementos do mapa.
8. Diseñar e implementar un sistema de xestión de usuarias e control de acceso, baseado en autenticación e autorización mediante roles que regulen os permisos de cada participante.
9. Desenvolver un perfil de usuaria que permita xestionar información persoal, preferencias de visualización e acceso aos mapas creados ou compartidos.
10. Implementar funcionalidade *offline*, permitindo a descarga previa de información para a consulta e edición de datos mesmo en ausencia de conexión a internet.
11. Garantir a seguridade e privacidade da información mediante mecanismos de autenticación, control de acceso e cifrado das comunicacións.

2. Descrición

2.1. Descrición xeral do problema

Na actualidade existen numerosas plataformas que permiten visualizar e interactuar con mapas dixitais a través da web. Estas ferramentas utilízanse habitualmente para localizar lugares, planificar desprazamentos ou consultar información xeográfica. Con todo, moitas destas solucións céntranse principalmente na navegación individual e na consulta de información, e non proporcionan mecanismos completos para a creación e xestión de mapas personalizados de forma colaborativa entre múltiples usuarias.

Ademais, unha gran parte das plataformas máis utilizadas dependen de servizos centralizados pertencentes a grandes empresas tecnolóxicas, o que implica a utilización de APIs privativas e a cesión de determinados datos de uso por parte das usuarias. Esta dependencia tecnolóxica limita a capacidade de control sobre a información xerada e dificulta a creación de solucións abertas que poidan ser adaptadas ou ampliadas segundo necesidades específicas.

Por outra banda, as ferramentas existentes adoitan presentar limitacións no ámbito da colaboración estruturada. Aínda que algúns servizos permiten compartir mapas ou localizacións, non adoitan ofrecer sistemas avanzados de permisos, edición colaborativa organizada ou historial detallado de cambios, funcionalidades que resultan necesarias cando varias persoas participan na construción e mantemento dun mesmo mapa.

Ante esta situación, xorde a necesidade de desenvolver unha aplicación que permita crear e xestionar mapas colaborativos de forma flexible, controlada e baseada en tecnoloxías abertas, proporcionándolles ás usuarias ferramentas para engadir puntos de interese, organizar información mediante categorías e colaborar con outros participantes segundo distintos niveis de permisos. O proxecto presentado aborda este problema mediante o desenvolvemento dunha aplicación web que integra mapas base de OpenStreetMap, incorpora funcionalidades colaborativas e garante o control das usuarias sobre a información xeoespacial que crean e comparten.

2.2. Requisitos funcionais

Xestión de usuarias

RF-01. Rexistro de usuarias. O sistema debe permitir o rexistro de novas usuarias mediante a introdución de información básica de identificación.

RF-02. Autenticación de usuarias. O sistema debe permitir que as usuarias rexistradas inicien sesión para acceder ás funcionalidades da aplicación.

RF-03. Activación da conta mediante correo electrónico. O sistema debe garantir que tras o rexistro se envía un token ao correo electrónico como requisito para o inicio de sesión.

RF-04. Xestión do perfil de usuaria. O sistema debe permitirlles ás persoas usuarias modificar a información do seu perfil, incluíndo nome, correo electrónico, nome de usuaria, contrasinal e preferencia de idioma da interface.

RF-05. Preferencias de idioma. O sistema debe permitir gardar e modificar a preferencia de idioma da interface, persistíndoa localmente e sincronizándoa co perfil da usuaria no servidor.

Xestión de mapas colaborativos

RF-06. Creación de mapas. O sistema debe permitirlles ás usuarias rexistradas crear novos mapas especificando nome, descrición e configuración de visibilidade.

RF-07. Consulta de mapas. O sistema debe permitirlles ás persoas usuarias consultar a lista de mapas aos que teñen acceso, organizados en tres categorías diferenciadas: mapas creados pola usuaria, mapas nos que colabora e mapas gardados.

RF-08. Edición de mapas. O sistema debe permitir modificar a información e configuración dos mapas creados pola usuaria.

RF-09. Eliminación de mapas. O sistema debe permitir eliminar mapas creados pola usuaria.

RF-10. Configuración de visibilidade dos mapas. O sistema debe permitir definir se un mapa é público ou privado.

RF-11. Visualización cartográfica. O sistema debe permitir visualizar mapas base e elementos xeográficos mediante unha interface cartográfica interactiva.

RF-12. Convite a usuarias. O sistema debe permitir convidar a outras usuarias a participar nun mapa público ou privado mediante un mecanismo de invitación.

RF-13. Xestión de roles por mapa. O sistema debe permitir asignarlles diferentes roles ás usuarias dentro dun mapa colaborativo.

RF-14. Control de acceso aos mapas. O sistema debe restrinxir a visualización e modificación dos mapas en función dos permisos asignados a cada usuaria.

RF-15. Gardado de mapas. O sistema debe permitirlle á usuaria gardar mapas públicos, co obxectivo de facilitar a consulta dos seus mapas públicos favoritos dos que non é colaboradora.

Xestión de puntos de interese

RF-16. Creación de puntos de interese. O sistema debe permitir engadir puntos de interese ao mapa indicando a súa localización xeográfica segundo os permisos da usuaria sobre o mapa.

RF-17. Modificación de puntos de interese. O sistema debe permitir editar a información asociada aos puntos de interese segundo os permisos da usuaria sobre o mapa.

RF-18. Eliminación de puntos de interese. O sistema debe permitir eliminar puntos de interese segundo os permisos da usuaria sobre o mapa.

Sistema de categorías

RF-19. Clasificación por categorías. O sistema debe permitir asignar unha categoría a cada punto de interese.

RF-20. Personalización visual das categorías. O sistema debe permitir definir atributos visuais das categorías, como cor ou icona.

RF-21. Filtrado por categorías. O sistema debe permitir consultar e filtrar os puntos de interese segundo a categoría.

Preferencias de visualización

RF-22. Activación e desactivación de capas. O sistema debe permitirlles ás persoas usuarias activar ou desactivar a visualización de determinadas categorías no mapa.

RF-23. Gardado do estado de visualización. O sistema debe permitir gardar as preferencias de visibilidade das capas para cada usuaria.

Notas en marcadores

RF-24. Creación de notas. O sistema debe permitir engadir anotacións ou comentarios asociados a marcadores do mapa. *(Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto).*

RF-25. Control de visibilidade das notas. O sistema debe permitir definir se unha nota é privada ou visible para outras usuarias do mapa. *(Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto).*

Historial de cambios

RF-26. Rexistro de accións. O sistema debe rexistrar automaticamente as accións realizadas sobre os mapas e os seus elementos.

RF-27. Consulta do historial. O sistema debe permitir consultar o historial de cambios segundo criterios como usuaria, data ou tipo de acción.

RF-28. Restauración de versións. O sistema debe permitir restaurar versións anteriores dos elementos modificados. *(Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto).*

Funcionalidade offline

RF-29. Acceso sen conexión. O sistema debe permitir acceder á información previamente almacenada no dispositivo mesmo sen conexión a internet. *(Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto).*

2.3. Requisitos non funcionais

Seguridade e privacidade

RNF-01. Protección de credenciais. O sistema debe almacenar as contrasinais das usuarias utilizando algoritmos de hash seguros.

RNF-02. Comunicación segura. O sistema debe garantir que todas as comunicacións entre cliente e servidor se realicen mediante conexións cifradas.

RNF-03. Protección de datos persoais. O sistema debe garantir a protección da información persoal das usuarias conforme á normativa vixente en materia de protección de datos.

RNF-04. Control de acceso. O sistema debe implementar mecanismos de control de acceso que restrinxan as accións das usuarias segundo os seus permisos.

RNF-05. Minimización de datos. O sistema debe evitar o almacenamento permanente de información innecesaria ou potencialmente sensible relacionada coa actividade das usuarias.

RNF-06. Ausencia de rastreadores externos. O sistema non debe empregar cookies de terceiros nin mecanismos de rastrexo externos que poidan comprometer a privacidade das persoas usuarias.

Usabilidade

RNF-07. Interface intuitiva. A aplicación debe proporcionar unha interface clara e comprensible que facilite o uso das funcionalidades principais sen requirir coñecementos técnicos avanzados.

RNF-08. Deseño responsivo. A interface debe adaptarse correctamente a diferentes tamaños de pantalla, incluíndo dispositivos móbiles e ordenadores de escritorio.

RNF-09. A interface debe seguir recomendacións básicas das pautas WCAG para garantir a accesibilidade.

RNF-10. Navegación eficiente. O sistema debe permitir unha navegación fluída entre as diferentes seccións da aplicación.

RNF-11. Soporte multilingüe. A aplicación debe permitir a utilización da interface en diferentes idiomas.

Rendemento

RNF-12. Tempo de resposta da API. Non debería superar os 2 segundos en condicións normais de uso.

RNF-13. Carga eficiente de datos cartográficos. O sistema debe optimizar a carga de mapas e elementos xeográficos para minimizar o tempo de visualización.

RNF-14. Xestión eficiente do almacenamento no cliente. O sistema debe xestionar de forma eficiente os datos almacenados no cliente, incluíndo as preferencias de usuaria persistidas en localStorage e os tokens de autenticación mantidos en memoria durante a sesión, evitando un uso innecesario dos recursos do dispositivo.

Escalabilidade e mantemento

RNF-15. Arquitectura modular. O sistema debe estar estruturado nunha arquitectura modular que facilite a evolución e ampliación da aplicación.

RNF-16. Separación entre cliente e servidor. A aplicación debe manter unha separación clara entre o *frontend* e o *backend* mediante unha API definida.

RNF-17. Calidade do código. O código do sistema debe seguir boas prácticas de desenvolvemento que faciliten a súa comprensión, mantemento e evolución.

RNF-18. Control de versións. O desenvolvemento do proxecto debe realizarse utilizando sistemas de control de versións para garantir o seguimento dos cambios.

Compatibilidade

RNF-19. Compatibilidade con navegadores modernos. A aplicación debe funcionar correctamente nos principais navegadores web modernos que implementen os estándares actuais da web.

RNF-20. Compatibilidade con estándares cartográficos abertos. O sistema debe ser compatible cos estándares utilizados polos mapas base de OpenStreetMap e tecnoloxías relacionadas.

RNF-21. Independencia de servizos propietarios. A aplicación debe evitar a dependencia de APIs ou servizos propietarios sempre que existan alternativas abertas equivalentes.

2.3. Análise de usuaria

Os tipos de usuarias que poden interactuar coa aplicación distínguense en dous niveis. Por unha banda, existen roles a nivel global que definen o tipo de usuaria dentro do sistema. Por outra banda, establécense roles específicos asociados a cada mapa colaborativo, que permiten asignar permisos concretos aos usuarias participantes.

Esta separación permite aplicar un control de acceso máis flexible, xa que unha mesma usuaria pode ter diferentes niveis de permiso segundo o mapa no que participe.

Tipos de usuaria a nivel de sistema

1. Visitante (usuaria anónima)

Usuaria que accede á aplicación sen autenticación.

Capacidades principais:

- Consultar mapas públicos.
- Navegar polo contido cartográfico dispoñible.
- Visualizar puntos de interese e categorías visibles.

Non pode crear nin modificar contido.

2. Usuaria rexistrada

Usuaria que dispón dunha conta no sistema e accede mediante autenticación.

Capacidades principais:

- Crear e xestionar mapas propios.
- Consultar mapas públicos.
- Participar en mapas colaborativos aos que foi convidada.
- Engadir contido segundo os permisos asignados.

3. Administradora da aplicación

Usuaria con privilexios administrativos a nivel global.

Capacidades principais:

- Supervisar o funcionamento da aplicación.
- Xestionar usuarias.
- Intervir en mapas en caso de problemas ou incidencias.

Tipos de usuarias a nivel de mapa

1. Membro

Usuaria con permisos básicos de consulta.

Capacidades:

- Visualizar o mapa.
- Consultar marcadores e categorías.

2. Colaboradora

Usuaria con permisos de edición sobre o contido do mapa.

Capacidades:

- Engadir marcadores.
- Modificar marcadores existentes.
- Engadir notas sobre os marcadores.

3. Administradora do mapa

Usuaria responsable da xestión do mapa colaborativo.

Capacidades:

- Modificar configuracións do mapa.
- Xestionar membros e permisos.
- Eliminar marcadores.
- Restaurar versións anteriores mediante o historial.

2.4. Casos de uso

CU-01 Visualizar mapas públicos

Actores: Visitante, usuaria rexistrada.

Descrición: Permite consultar mapas configurados como públicos sen necesidade de autenticación.

Precondicións: O mapa debe estar configurado como público.

Fluxo principal:

1. A usuaria accede á lista de mapas públicos.

2. O sistema amosa os mapas dispoñíbles.
3. A usuaria selecciona un mapa.
4. O sistema carga o mapa coas capas e puntos de interese asociados.

Resultado: A usuaria visualiza o mapa e pode interactuar cos elementos visibles.

CU-02 Crear mapa

Actor: Usuaria rexistrada.

Descrición: Permite crear un novo mapa configurable polo usuaria.

Precondicións: A usuaria debe estar autenticada.

Fluxo principal:

1. A usuaria accede á opción de creación de mapas.
2. Introduce os datos do mapa (nome, descrición, visibilidade).
3. O sistema valida a información.
4. O sistema crea o novo mapa e gárdao na base de datos.

Resultado: O mapa queda asociado á usuaria creadora.

CU-03 Xestionar permisos dun mapa

Actor: Administradora do mapa.

Descrición: Permite xestionar os permisos dos usuarios que participan nun mapa colaborativo.

Precondicións: A usuaria debe ser administradora do mapa.

Fluxo principal:

1. A administradora accede á sección de xestión de membros.
2. Selecciona unha usuaria existente ou envía un convite.
3. Define o rol correspondente (membro, colaboradora, administradora).

4. O sistema garda os cambios.

Resultado: Os permisos da usuaria quedan actualizados.

CU-04 Rexistro e autenticación

Actor: Visitante.

Descrición: Permite a creación dunha conta de usuaria e o acceso posterior ao sistema mediante autenticación.

Fluxo principal:

1. A persoa visitante accede á páxina de rexistro.
2. Introduce os datos necesarios para crear unha conta.
3. O sistema valida a información introducida.
4. O sistema crea a conta de usuaria.
5. A usuaria pode iniciar sesión na aplicación.

Resultado: Créase unha nova conta e a usuaria pode acceder ás funcionalidades da aplicación.

CU-05 Edición colaborativa de mapas

Actores: Colaboradora, Administradora do mapa.

Descrición: Permite modificar o contido dun mapa colaborativo segundo os permisos asignados.

Precondicións:

- A usuaria debe estar autenticada.
- A usuaria debe ter permisos de edición no mapa.

Fluxo principal:

1. A usuaria accede ao mapa colaborativo.
2. Selecciona un elemento do mapa ou crea un novo.

3. Realiza as modificacións necesarias.
4. O sistema valida os cambios.
5. O sistema garda a modificación e rexistra a acción no historial.

Resultado: Os cambios quedan almacenados e visibles para os demais usuarios con acceso ao mapa.

CU-06 Engadir marcador

Actores: Colaboradora, Administradora do mapa.

Descrición: Permite engadir un novo marcador nun mapa colaborativo, especificando a súa localización xeográfica e a categoría asociada.

Precondicións:

- A usuaria debe estar autenticada no sistema.
- A usuaria debe ter permisos de edición no mapa correspondente.
- O mapa debe estar cargado na interface de visualización.

Fluxo principal:

1. A usuaria accede a un mapa no que ten permisos de edición.
2. A usuaria selecciona a opción de engadir marcador na interface do mapa.
3. A usuaria indica a localización do marcador (premendo sobre o mapa).
4. O sistema solicita os datos do marcador (nome, categoría, descrición opcional ou outros atributos).
5. A usuaria introduce a información requirida.
6. O sistema valida os datos introducidos.
7. O sistema garda o marcador na base de datos asociado o marcador ao mapa e á categoría seleccionada.
8. O sistema rexistra a acción no historial de cambios do mapa.

Resultado: O novo marcador aparece representado no mapa e pasa a estar dispoñible para todas as usuarias con acceso ao mapa segundo os permisos establecidos.

CU-07 Filtrar marcadores por categoría

Actores: Visitante, usuaria rexistrada, Membro, Colaboradora, Administradora do mapa.

Descrición: Permite amosar ou ocultar no mapa os marcadores pertencentes a determinadas categorías para facilitar a exploración e visualización da información.

Precondicións: A usuaria debe estar visualizando un mapa que conteña marcadores asociados a categorías.

Fluxo principal:

1. A usuaria accede a un mapa na aplicación.
2. O sistema amosa as categorías dispoñibles asociadas aos marcadores do mapa.
3. A usuaria selecciona unha ou varias categorías para activar ou desactivar a súa visualización.
4. O sistema actualiza a visualización do mapa en tempo real.
5. Os marcadores pertencentes ás categorías desactivadas deixan de amosarse no mapa.

Resultado: O mapa amósase filtrado segundo as categorías seleccionadas, permitíndole á usuaria visualizar un subconxunto específico de marcadores.

Postcondición: Se a usuaria está autenticada, o sistema pode gardar as preferencias de visibilidade das categorías para futuras sesións.

3. Alcance

O proxecto consiste no deseño, desenvolvemento e despregamento dunha aplicación web colaborativa para a creación, edición e xestión de mapas personalizados. A aplicación estará baseada exclusivamente en tecnoloxías libres e terá como principio fundamental a protección da privacidade das persoas usuarias e o control sobre os seus datos.

O sistema desenvolverase seguindo unha arquitectura cliente-servidor, composta por un *frontend* web interactivo e un *backend* encargado da lóxica de negocio, autenticación e persistencia dos datos. A comunicación entre ambos compoñentes realizarase mediante unha API REST segura. O sistema empregará unha base de datos relacional con soporte xeoespacial para o almacenamento de información cartográfica.

O alcance do proxecto inclúe tanto o desenvolvemento funcional da aplicación como o deseño da arquitectura técnica, o modelado de datos, a implementación da API e a preparación dun entorno de execución que permita o despregamento do sistema.

3.1. Funcionalidades incluídas

Xestión de usuarias e autenticación

O sistema permitirá a creación e xestión de contas de usuaria mediante un mecanismo de rexistro e autenticación seguro.

Entre as funcionalidades incluídas atópanse:

- Rexistro de novos usuarias.
- Inicio de sesión mediante autenticación baseada en *tokens*.
- Xestión do perfil de usuaria.
- Sistema de roles que determina os permisos de acceso aos recursos da aplicación.

Sistema de mapas colaborativos

A aplicación permitirá a creación e xestión de mapas personalizados que poden ser utilizados de forma individual ou colaborativa.

As funcionalidades principais inclúen:

- Creación de mapas públicos ou privados.
- Xestión da información básica de cada mapa (nome, descrición e configuración).
- Sistema de permisos que permite asignar diferentes niveis de acceso aos participantes dun mapa.
- Posibilidade de convidar outros usuarios a mapas privados mediante enlaces de acceso con caducidade.

Edición de contido xeoespacial

Os usuarios con permisos adecuados poderán modificar o contido dun mapa mediante a edición de elementos xeográficos.

Entre as accións dispoñibles inclúense:

- Engadir puntos de interese mediante coordenadas xeográficas.
- Modificar ou eliminar elementos existentes.
- Organizar os puntos mediante categorías configurables.
- Asociar atributos visuais aos elementos, como cores ou iconas.

Historial de cambios

O sistema incorporará un mecanismo de seguimento que permita rexistrar as modificacións realizadas sobre os mapas colaborativos.

Este historial permitirá:

- Consultar accións realizadas polos usuarios.
- Identificar cambios segundo data, usuario e tipo de operación.
- Facilitar a recuperación de versións anteriores do contido.

Preferencias e personalización

Cada usuaria poderá configurar determinados aspectos da visualización da aplicación.

Entre as opcións incluídas atópanse:

- Activar ou desactivar capas ou categorías do mapa.
- Gardar preferencias de visualización.
- Acceder rapidamente a mapas recentes ou favoritos.

Funcionalidade *offline*

A aplicación incorporará soporte para funcionamento parcial sen conexión mediante o uso de tecnoloxías de almacenamento local no navegador.

Isto permitirá:

- Consultar mapas previamente descargados.
- Acceder a puntos de interese almacenados localmente.
- Sincronizar os datos cando se restableza a conexión á rede.

3.2. Aspectos técnicos incluídos no proxecto

Ademais das funcionalidades descritas, o proxecto inclúe o desenvolvemento da infraestrutura técnica necesaria para o funcionamento da aplicación:

- Implementación dun *backend* baseado nun framework de desenvolvemento web para Java.
- Desenvolvemento dun *frontend* web interactivo baseado en tecnoloxías modernas de JavaScript.
- Deseño dunha API REST para a comunicación entre cliente e servidor.
- Modelado dunha base de datos relacional con soporte para datos xeoespaciais.
- Implementación de medidas de seguridade e protección da privacidade dos usuarios.

3.3. Limitacións do alcance

Determinadas funcionalidades identificadas durante a fase de análise quedaron fóra do alcance da implementación actual por razóns de viabilidade temporal e complexidade técnica, priorizando aquelas funcionalidades que constitúen o núcleo colaborativo da aplicación. A continuación descríbense estas limitacións, xunto coa súa xustificación.

Notas en marcadores

Aínda que os requisitos RF-22 e RF-23 contemplan a posibilidade de engadir anotacións ou comentarios asociados a marcadores con control de visibilidade, esta funcionalidade non foi implementada na versión actual. Os marcadores xa incorporan un campo de descrición que cobre a necesidade básica de asociar información textual a un punto de interese, polo que se considerou que a implementación dunha capa de notas independente, con dobre nivel de visibilidade (privada ou compartida) e un modelo de datos propio, non era prioritaria respecto a outras funcionalidades do núcleo colaborativo. Queda identificada como liña de mellora para versións posteriores.

Restauración de versións no historial de cambios

O sistema rexistra automaticamente as accións realizadas sobre os mapas e os seus elementos (RF-24 e RF-25), e permite consultar o historial con filtros por tipo de acción e por usuario. Non obstante, a restauración de versións anteriores dos elementos modificados (RF-26) non está implementada. Esta funcionalidade implica, ademais da lóxica de reversión, unha estratexia de almacenamento do estado previo de cada elemento en cada acción, o que supón un alcance técnico significativamente maior. A capacidade de consulta do historial cobre o caso de uso principal de trazabilidade; a reversión queda como ampliación futura.

Funcionalidade offline

A integración de acceso sen conexión (RF-27) mediante Service Workers e almacenamento local con IndexedDB non foi implementada nesta versión. A funcionalidade *offline* require un deseño específico de sincronización bidireccional de datos, resolución de conflitos en caso de edicións concorrentes e xestión do ciclo de vida da caché, o que a converte na funcionalidade de maior complexidade técnica do proxecto. A súa implementación completa e robusta non é compatible co período de desenvolvemento dispoñible. A arquitectura da aplicación, baseada nunha SPA con API REST stateless, non presenta obstáculos estruturais para incorporar esta funcionalidade en versións futuras.

Capacidades de administración global

O sistema inclúe o rol ADMIN a nivel de aplicación, mais non se implementaron os endpoints nin a interface de administración global. As funcionalidades propias deste rol —xestión de contas de usuaria, supervisión de mapas públicos e intervención en caso de incidencias— non se aliñan coas funcionalidades colaborativas principais do proxecto e foron descartadas para esta versión en favor delas. Queda reservada para versións futuras, onde poida deseñarse de forma coherente con requisitos de seguridade e usabilidade axeitados.

Consultas xeoespaciais nativas

As coordenadas dos marcadores e mapas almacénanse como valores numéricos de latitude e lonxitude. Aínda que o proxecto declara a dependencia de PostGIS e emprega PostgreSQL como base de datos de produción, a busca por radio utiliza a fórmula de Haversine calculada en memoria na capa de servizo Java, en lugar de delegar esta operación en consultas espaciais nativas de PostgreSQL. Esta decisión permite un desenvolvemento máis áxil durante o prototipo, pero introduce limitacións de rendemento con volumes grandes de datos. A migración a consultas PostGIS nativas queda identificada como mellora técnica para versións posteriores.

Paxinación de listaxes

Ningunha das listaxes da aplicación —mapas, marcadores, categorías, historial de cambios ou membros— implementa paxinación. Todos os resultados se cargan nun único request. Esta decisión simplifica o desenvolvemento durante o período de prototipado, pero limita a escalabilidade da aplicación ante volumes de datos elevados. A incorporación de paxinación ou carga por demanda queda como mellora técnica pendente.

4. Planificación

4.1. Metodoloxía de desenvolvemento

Para organizar o desenvolvemento do proxecto utilízase unha metodoloxía de traballo áxil baseada nos principios de Scrum, adaptada ás características dun proxecto individual de fin de ciclo.

Scrum é un marco de traballo orientado ao desenvolvemento iterativo e incremental, no que o produto evoluciona progresivamente mediante entregas parciais que permiten validar as funcionalidades implementadas e detectar posibles problemas de forma temperá.

A aplicación deste enfoque permite dividir o desenvolvemento en iteracións controladas, facilitar a planificación do traballo e manter unha visión clara do progreso do proxecto ao longo do tempo.

Metodoloxía áxil adaptada

A metodoloxía empregada baséase nunha versión simplificada de Scrum, estruturada en sprints ou iteracións de desenvolvemento, cada unha cun conxunto de tarefas definidas, un período temporal concreto e obxectivos funcionais claros.

Para organizar o traballo utilizáronse os principais artefactos de Scrum:

- **Product Backlog**, que recolle o conxunto de funcionalidades e requisitos do sistema, organizados en forma de historias de usuaria.
- **Sprint Backlog**, que contén as tarefas seleccionadas para cada iteración de desenvolvemento.
- **Incremento de produto**, correspondente ás funcionalidades implementadas e operativas ao final de cada sprint.

Este enfoque permitiu:

- Dividir o proxecto en partes asumibles e organizadas.
- Priorizar as funcionalidades máis importantes do sistema.
- Avaliar o progreso de forma continua ao longo do desenvolvemento.

- Introducir melloras ou modificacións de maneira controlada.
- Manter o foco nos obxectivos principais do proxecto.

Cada sprint incluíu tarefas de análise, deseño, implementación, probas e documentación, permitindo producir incrementos funcionais da aplicación en cada iteración.

Seguimento e control do progreso

Ao finalizar cada sprint realizouse unha revisión do traballo desenvolvido co obxectivo de comprobar o cumprimento das funcionalidades planificadas, identificar posibles problemas ou dificultades técnicas e actualizar e reorganizar o backlog do proxecto para as seguintes iteracións.

Este proceso de revisión continua permitiu manter unha visión clara do estado do proxecto e asegurar que o desenvolvemento se mantivese aliñado cos obxectivos definidos inicialmente.

4.2. Organización en sprints

O desenvolvemento do proxecto estruturouse en iteracións curtas denominadas sprints, cada unha cunha duración aproximada dunha semana. Esta duración seleccionouse para adaptarse ao calendario académico e permitir unha evolución progresiva do sistema mediante incrementos funcionais frecuentes.

Cada sprint inclúe tarefas de análise, deseño, implementación, probas e documentación. Ao finalizar cada iteración obtense un prototipo funcional do sistema, que incorpora as novas funcionalidades desenvolvidas durante ese período.

Este enfoque permite validar progresivamente o comportamento da aplicación e detectar posibles problemas de integración entre os diferentes compoñentes do sistema.

4.3. Planificación detallada dos sprints

A planificación estrutúrase en trece sprints, incluíndo un sprint inicial de preparación e análise.

Sprint 0 — Análise e preparación

Data de inicio: 02/03/2026.

Data de fin: 08/03/2026.

Obxectivo: Establecer as bases conceptuais e técnicas do proxecto, definindo a arquitectura do sistema, os requisitos iniciais e a planificación do desenvolvemento.

Tarefas:

- Definición do alcance do proxecto.
- Análise dos requisitos funcionais e non funcionais.
- Definición da metodoloxía de traballo.
- Modelado conceptual do sistema.
- Deseño preliminar da arquitectura cliente-servidor.
- Análise de viabilidade técnica das tecnoloxías seleccionadas.
- Preparación da infraestrutura inicial do proxecto.

Entregables:

- Primeiro borrador dos cinco primeiros apartados da memoria.
- Diagrama Entidade Relación.
- Diagrama de clases.
- Wireframes da aplicación.
- Guía de estilos.

Sprint 1 — Infraestrutura inicial

Data de inicio: 09/03/2026.

Data de fin: 15/03/2026.

Obxectivo: Preparar o entorno de desenvolvemento e a estrutura básica da aplicación.

Tarefas:

- Configuración dos entornos de traballo de desenvolvemento e produción.
- Configuración do *backend* en Spring Boot.

- Configuración do *frontend* en React.
- Configuración da base de datos PostgreSQL.
- Integración inicial con PostGIS.
- Estrutura inicial da API REST.

Entregables: Prototipo inicial da aplicación.

Sprint 2 — Sistema de usuarias

Data de inicio: 16/03/2026.

Data de fin: 22/03/2026.

Obxectivo: Implementar o sistema de rexistro e autenticación de usuarias.

Tarefas:

- Modelado das entidades de usuaria.
- Implementación do rexistro de usuarias.
- Implementación do inicio de sesión.
- Configuración de Spring Security.
- Implementación de autenticación mediante JWT.
- Creación dos primeiros endpoints da API de usuarias.
- Interface básica de rexistro e login no *frontend*.

Entregables: Prototipo con autenticación funcional.

Sprint 3 — Sistema de mapas

Data de inicio: 23/03/2026.

Data de fin: 29/03/2026.

Obxectivo: Implementar a estrutura básica de mapas colaborativos.

Tarefas:

- Modelado da entidade mapa.
- Creación de mapas.
- Edición e eliminación de mapas.
- Definición de visibilidade pública/privada.
- Implementación dos endpoints de xestión de mapas.
- Integración de Leaflet para visualización cartográfica.

Entregables:

- Prototipo con sistema de mapas funcional.

Sprint 4 — Sistema de puntos de interese

Data de inicio: 30/03/2026.

Data de fin: 05/04/2026.

Obxectivo: Permitir a creación e edición de elementos xeográficos dentro dun mapa.

Tarefas:

- Modelado de marcadores.
- Creación de endpoints para marcadores.
- Integración con PostGIS.
- Interface para engadir marcadores no mapa.
- Edición e eliminación de marcadores.

Entregables: Prototipo con edición de contido xeoespacial.

Sprint 5 — Categorias e atributos visuais

Data de inicio: 06/04/2026.

Data de fin: 12/04/2026.

Obxectivo: Organizar os puntos mediante categorías configurables.

Tarefas:

- Modelado da entidade categoría.
- Asociación de categorías a marcadores.
- Configuración de cores e iconas.
- Implementación de filtros por categoría.
- Interface de xestión de categorías.

Entregables: Prototipo con categorización de contido.

Sprint 6 — Sistema de permisos

Data de inicio: 13/04/2026.

Data de fin: 19/04/2026.

Obxectivo: Implementar o modelo de permisos para mapas colaborativos.

Tarefas:

- Modelado de roles por mapa.
- Implementación de relación usuaria-mapa.
- Control de acceso aos endpoints.
- Interface de xestión de membros.
- Implementación de convites con *token*.

Entregables: Prototipo con colaboración entre usuarias

Sprint 7 — Historial de cambios

Data de inicio: 20/04/2026.

Data de fin: 26/04/2026.

Obxectivo: Implementar o sistema de seguimento de cambios en mapas, marcadores e categorías.

Tarefas:

- Modelado da entidade historial.
- Rexistro automático de accións.
- Consulta do historial.
- Interface de visualización do historial.

Entregables: Prototipo con trazabilidade de modificacións.

Sprint 8 — Personalización do usuaria

Data de inicio: 27/04/2026.

Data de fin: 03/05/2026.

Obxectivo: Implementar preferencias de visualización e personalización.

Tarefas:

- Modelado de preferencias.
- Filtros de visibilidade de categorías.
- Gardado de configuración de usuaria.
- Interface de configuración.

Entregables: Prototipo con personalización do mapa.

Sprint 9 — Mellora da interface e rendemento

Data de inicio: 04/05/2026.

Data de fin: 10/05/2026.

Obxectivo: Mellorar a usabilidade e optimizar o rendemento do sistema.

Tarefas:

- Optimización de consultas xeoespaciais.
- Mellora da interface de usuaria.
- Optimización da carga de datos.
- Corrección de erros detectados.

Entregables: Prototipo estable da aplicación.

Sprint 10 — Funcionalidade *offline*

Data de inicio: 11/05/2026.

Data de fin: 17/05/2026.

Obxectivo: Implementar soporte para uso sen conexión.

Tarefas:

- Implementación de Service Workers.
- Almacenamento local mediante IndexedDB.
- Sistema de descarga de datos.
- Sincronización ao recuperar conexión.

Entregables: Prototipo con soporte *offline*.

Sprint 11 — Probas e estabilización

Data de inicio: 18/05/2026.

Data de fin: 24/05/2026.

Obxectivo: Validar o funcionamento global da aplicación e corrixir erros.

Tarefas:

- Probas funcionais.
- Probas de seguridade.

- Corrección de erros.
- Optimización final do sistema.

Entregables: Versión estable da aplicación.

Sprint 12 — Documentación e despregamento

Data de inicio: 25/05/2026.

Data de fin: 31/05/2026.

Obxectivo: Preparar a documentación final do proxecto e realizar o despregamento da aplicación.

Tarefas:

- Redacción da memoria técnica.
- Preparación da documentación da API.
- Configuración de Docker.
- Preparación do despregamento.
- Elaboración da presentación final.

Entregables:

- Versión final da aplicación.
- Manual técnico.
- Manual de usuaria.
- Instrucións de despregamento do sistema.
- Versión final da memoria.

4.4. Fitos do proxecto

Fito	Descrición	Sprint
Definición da arquitectura	Establecemento da arquitectura do sistema e planificación inicial	Sprint 0
Infraestrutura base	Configuración do entorno de desenvolvemento e estrutura inicial da aplicación	Sprint 1
Sistema de autenticación	Implementación do rexistro e inicio de sesión de usuarias	Sprint 2
Sistema de mapas	Creación e visualización de mapas colaborativos	Sprint 3
Edición xeoespacial	Implementación de puntos de interese e edición de contido	Sprint 4
Sistema de permisos	Xestión de roles e colaboración entre usuarias	Sprint 6
Sistema de historial	Rexistro de cambios e trazabilidade	Sprint 7
Funcionalidade <i>offline</i>	Implementación de acceso sen conexión	Sprint 10
Versión estable	Finalización das probas e estabilización do sistema	Sprint 11
Versión final	Documentación e despregamento do sistema	Sprint 12

4.5. Diagrama de Gantt

5. Medios a utilizar

5.1. Tecnoloxías

O desenvolvemento da aplicación baséase nun conxunto de tecnoloxías de código aberto amplamente utilizadas no desenvolvemento de aplicacións web modernas. A selección realizouse atendendo a criterios de robustez, seguridade, madurez do ecosistema e compatibilidade cos principios de software libre e respecto pola privacidade das persoas usuarias.

A arquitectura do sistema segue un modelo cliente-servidor, no que o backend expón unha API REST encargada da lóxica de negocio e da xestión dos datos, mentres que o frontend proporciona unha interface web interactiva que permite visualizar e editar mapas colaborativos.

Backend

Java (OpenJDK). Linguaxe principal empregada para o desenvolvemento do backend. Escollida pola súa robustez, estabilidade e amplo ecosistema para a construción de aplicacións empresariais escalables.

Spring Framework + Spring Boot. Framework e módulo utilizados para o desenvolvemento da aplicación backend e da API REST. Permiten estruturar a aplicación en compoñentes modulares e facilitan a configuración e integración de distintos servizos.

Spring Security. Módulo de seguridade do ecosistema Spring empregado para implementar autenticación, autorización e control de acceso aos recursos da aplicación.

JWT (JSON Web Tokens). Tecnoloxía empregada para a autenticación baseada en tokens entre cliente e servidor, permitindo un modelo de comunicación sen estado (*stateless*).

Hibernate / JPA. Tecnoloxía de mapeo obxecto-relacional (ORM) utilizada para facilitar a persistencia de datos e a interacción entre os obxectos da aplicación e a base de datos relacional.

PostgreSQL. Sistema de xestión de bases de datos relacional de código aberto recoñecido pola súa estabilidade, rendemento e capacidade para xestionar grandes volumes de información.

PostGIS. Extensión xeoespacial de PostgreSQL que permite almacenar, consultar e procesar datos xeográficos como coordenadas, xeometrías e elementos cartográficos.

Frontend

React. Biblioteca JavaScript para a construción de interfaces de usuaria baseadas en compoñentes reutilizables, que permite desenvolver aplicacións web dinámicas e interactivas.

Leaflet.js. Biblioteca JavaScript lixeira para a visualización de mapas interactivos no navegador. Permite integrar mapas base de OpenStreetMap e representar elementos xeográficos como marcadores ou capas cartográficas.

Fetch API / Axios. Ferramentas utilizadas para realizar comunicacións HTTP asíncronas entre o frontend e a API REST do backend.

Tecnoloxías para soporte offline

Service Workers. Tecnoloxía do navegador que permite interceptar peticións de rede e implementar mecanismos de cache e funcionamento offline.

IndexedDB. Sistema de almacenamento local do navegador que permite gardar datos estruturados no cliente, utilizado para almacenar información necesaria cando a aplicación se utiliza sen conexión a internet.

Protocolos e seguridade

HTTPS. Protocolo de comunicación cifrado empregado para garantir a confidencialidade e integridade das comunicacións entre cliente e servidor.

5.2. Ferramentas

Durante o desenvolvemento do proxecto empregáronse diversas ferramentas de software destinadas a facilitar a implementación, organización e documentación da aplicación.

Desenvolvemento e control de versións

Git. Sistema de control de versións distribuído utilizado para rexistrar os cambios realizados no código fonte e facilitar o seguimento do desenvolvemento do proxecto.

GitHub. Plataforma de hospedaxe de repositorios Git empregada para almacenar o código do proxecto e xestionar as distintas versións do desenvolvemento.

Documentación e probas da API

Swagger / OpenAPI. Ferramentas utilizadas para documentar e probar os endpoints da API REST, facilitando a comprensión da interface de programación e a interacción entre cliente e servidor.

Despregamento

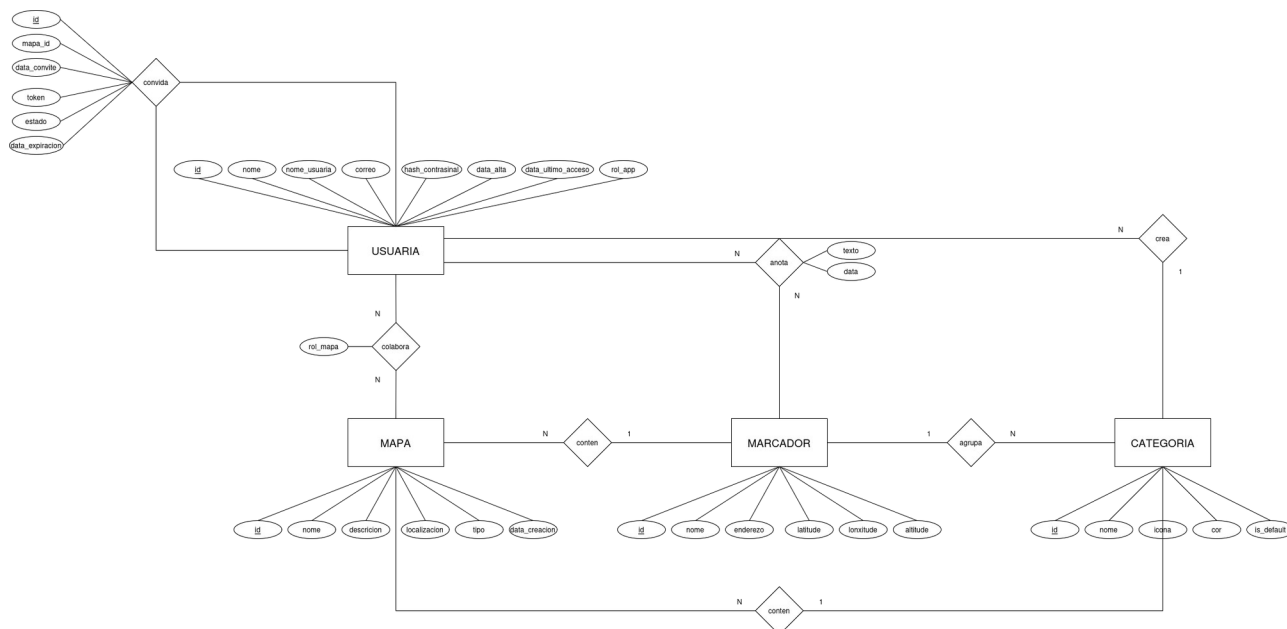
Docker. Tecnoloxía de contedorización empregada para empacar a aplicación e as súas dependencias, permitindo executar o sistema en diferentes entornos de maneira consistente.

Deseño e modelado

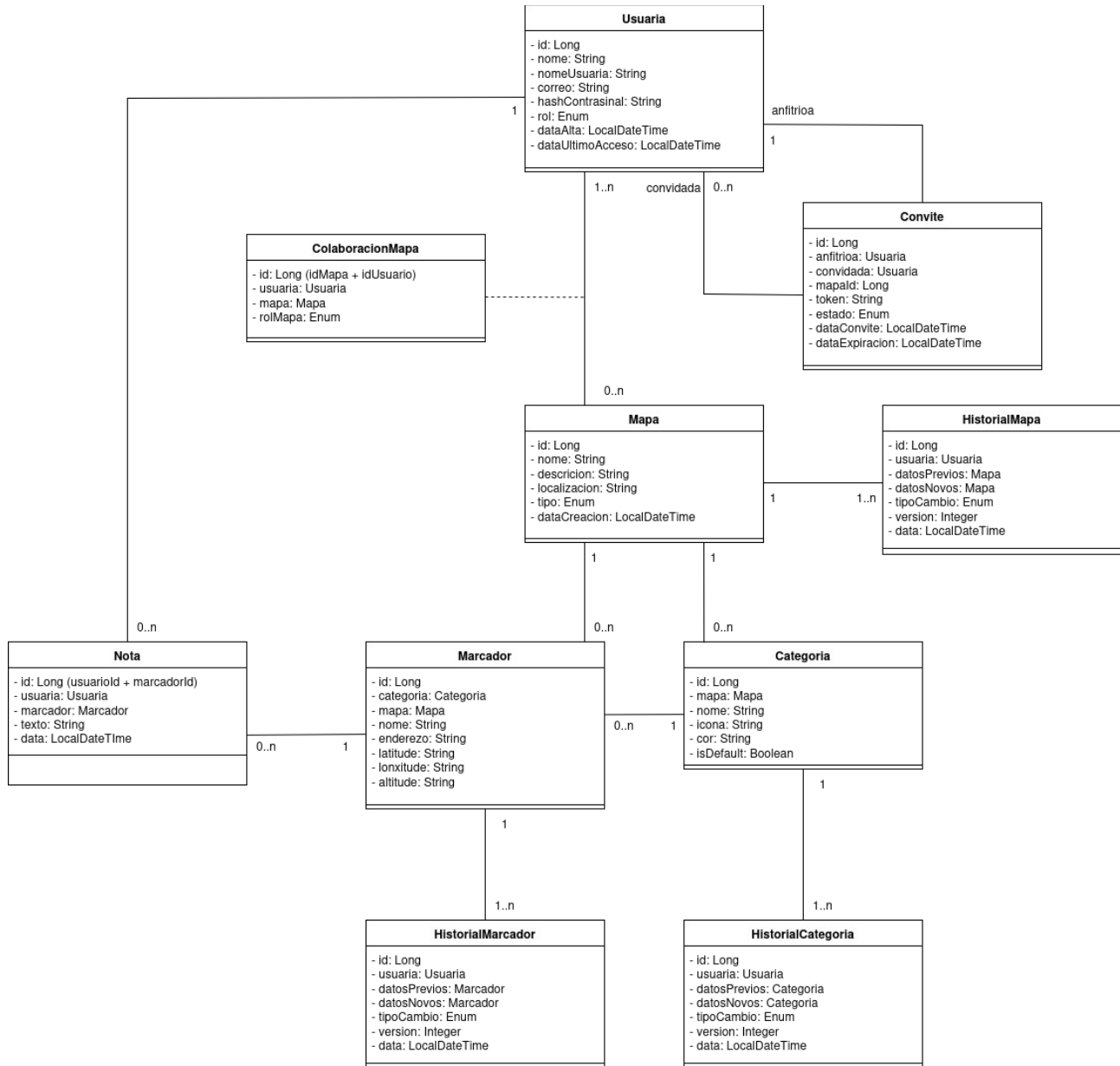
Draw.io. Ferramenta utilizada para a elaboración de diagramas técnicos, incluíndo diagramas UML, modelos de arquitectura e diagramas de fluxo.

Moqups. Ferramenta empregada para a creación de prototipos de interface e o deseño de *wireframes* da aplicación.

PRIMEIRA VERSIÓN DIAGRAMA ENTIDADE RELACIÓN



PRIMEIRA VERSIÓN DIAGRAMA DE CLASES UML



6. Presuposto

6.1. Custos estimados

7. Execución

7.1. Deseño do sistema

Arquitectura xeral da aplicación

A aplicación segue unha arquitectura cliente-servidor con separación completa entre o frontend e o backend, comunicados mediante unha API REST sobre HTTP. Esta separación permite que ambos os compoñentes evolucionen de forma independente e facilita o despregamento en contornos distintos.

O **backend** é unha aplicación Java construída con Spring Boot que expón unha API REST stateless. Xestiona a autenticación, a lóxica de negocio, o control de acceso e a persistencia dos datos. En desenvolvemento utiliza unha base de datos H2 en memoria; en produción, PostgreSQL.

O **frontend** é unha Single Page Application (SPA) construída con React e Vite. Comunícase co backend exclusivamente a través da API REST, xestiona o estado da sesión en memoria e renderiza a interface de xeito dinámico sen recargar a páxina.

//TODO facer que o fluxo de comunicación sexa unha imaxe, e facelo tamén para back

Navegador (React SPA)

| HTTP / JSON



API REST (Spring Boot)

| JPA / Hibernate



Base de datos (H2 dev / PostgreSQL prod)



Servizos externos:

- Nominatim (xeolocalización inversa)

- Mailpit / SMTP (envío de correos)

Modelo de datos

O modelo de datos xira arredor de tres entidades principais: Usuaria, Mapa e Marcador, complementadas por entidades de relación e soporte.

Entidades principais

Usuaria representa unha persoa rexistrada no sistema. Almacena as credenciais de acceso (contrasinal como hash bcrypt), o rol de aplicación (USER ou ADMIN), o idioma preferido e o estado de verificación do correo. Está relacionada cos mapas que crea, os mapas nos que participa como membro, os mapas que garda como favoritos e os convites enviados e recibidos.

Mapa é o elemento central da aplicación. Cada mapa ten un nome, unha descrición opcional, coordenadas xeográficas de referencia (latitude e lonxitude), datos de localización obtidos por xeolocalización inversa (cidade, rexión, país), e un tipo de visibilidade (PUBLICO ou PRIVADO). O campo creadoPor almacena o nome de usuaria da propietaria e determina os permisos máximos sobre o mapa.

Marcador representa un punto de interese dentro dun mapa. Ten nome, descrición opcional, coordenadas xeográficas propias, e pode estar asociado a unha categoría. O campo creadoPor permite aplicar a lóxica de permisos baseada no rol da usuaria dentro do mapa.

Entidades de relación

MapaMembro establece a relación entre unha usuaria e un mapa privado, almacenando o rol asignado (MEMBRO, COLABORADORA ou ADMIN_MAPA). É a entidade que habilita a colaboración estruturada.

Convite xestiona o proceso de incorporación a un mapa privado. Contén un token UUID único, as referencias á usuaria anfitrión e á convidada, o mapa de destino, o rol que se lle asignará ao aceptar, o estado do convite e a data de expiración. A expiración é *lazy*: o convite non se marca automaticamente como EXPIRADO por ningún proceso de fondo, senón no momento en que se intenta usar ou consultar.

MapaGardado implementa o sistema de favoritos: relaciona unha usuaria cun mapa público que desexa gardar para acceso rápido, sen implicar permisos de edición.

EntradaHistorial rexistra as accións realizadas sobre os elementos dun mapa. Almacena o tipo de acción (CREAR, EDITAR, ELIMINAR), o tipo de elemento afectado (MAPA, MARCADOR, CATEGORIA), o identificador e nome do elemento, o nome de usuaria que

realizou a acción, e un campo de detalle con información adicional. A usuaria almacénase como String (non como FK) para preservar o rexistro histórico aínda que a conta sexa eliminada.

Categoría permite organizar os marcadores dun mapa en grupos visuais, con nome, cor en formato hexadecimal e icona. A eliminación dunha categoría desvincula os marcadores asociados sen eliminalos.

Xestión da autenticación

RefreshToken almacena o hash do token de refresco activo de cada sesión, coa data de expiración e un indicador de revogación. O token real nunca se persiste; só o seu hash, comparado no momento da renovación.

TokenVerificacion xestiona os tokens de activación de conta enviados por correo electrónico, con data de expiración e indicador de uso.

Relacións e integridade referencial

A eliminación dun Mapa propaga en cascada a todos os seus elementos dependentes: categorías, marcadores, membros, gardados e convites. O EntradaHistorial elimínase mediante unha query @Modifying JPQL explícita, xa que a súa relación co mapa non usa cascade JPA estándar. A eliminación dunha Usuaria require un proceso secuencial que limpa primeiro as súas participacións en mapas alleos antes de eliminar os seus propios mapas.

Deseño da capa de servizo e lóxica de negocio

O backend segue un patrón de capas estrito: os Controller reciben e validan as peticións HTTP e delegan en Service; os Service conteñen toda a lóxica de negocio e de acceso; os Repository proporcionan acceso á base de datos mediante Spring Data JPA.

O control de acceso non usa anotacións @PreAuthorize nin roles de Spring Security nos endpoints. En cambio, realízase na capa de servizo mediante MapaAccesoService, un compoñente utilitario que centraliza a verificación de acceso e é reutilizado por CategoriaServiceImpl, MarcadorServiceImpl, HistorialServiceImpl e MapaMembroServiceImpl. Esta decisión mantén a lóxica de permisos nun único punto e evita duplicación entre servizos.

Os Service comunican erros ao Controller mediante excepcións semánticas: IllegalArgumentException para recursos non atopados (mapeada a HTTP 404) e IllegalStateException para operacións non permitidas (mapeada a HTTP 403). O

GlobalExceptionHandler (@RestControllerAdvice) captura estas excepcións e formata respostas de erro consistentes en JSON.

O mapeo entre entidades e DTOs realízase manualmente nos Service, sen uso de ferramentas de mapeo automático como MapStruct. Os Request DTO están anotados con Jakarta Validation para a validación de entrada; os Response DTO garanten que nunca se expón ningunha entidade JPA directamente ao cliente.

Deseño da interface de usuaria

A interface está organizada como unha SPA con dúas zonas diferenciadas: páxinas públicas (login, rexistro, verificación e mapa principal) e páxinas protexidas que requiren autenticación, accesibles tras pasar polo compoñente ProtectedRoute.

As páxinas protexidas comparten o AppLayout, que inclúe un Sidebar lateral con navegación principal, expandible e colapsable, e un BottomNav para dispositivos móbiles. A interface adapta os elementos dispoñibles segundo o rol da usuaria sobre cada mapa: o rol efectivo calcúlase no cliente comparando o campo creadoPor do mapa co nome de usuaria en sesión, e completando coa entrada en MapaMembro se existe.

A visualización cartográfica usa Leaflet con tiles de OpenStreetMap. O compoñente MapViewer opera en dous modos: só lectura (cando onLocationSelect é null) e modo de selección de localización (cando se está a crear ou editar un marcador). Un pin provisional mostra a posición seleccionada antes de confirmar. A busca de enderezos usa a API de Nominatim con debounce de 400 ms para limitar as peticións.

O estado global da aplicación xestiónase con dous mecanismos complementarios: AuthContext (React Context) para o estado de sesión, e catro stores Zustand para estado de UI persistente (sidebar, idioma) e estado transitorio (notificacións, explorador de mapas públicos).

Deseño da seguridade e control de acceso

A autenticación baséase en JWT con dous tokens: un accessToken de curta duración (15 minutos en desenvolvemento) e un refreshToken de 7 días con rotación. Cada vez que se usa o refreshToken para renovar a sesión, o token antigo revógase e xérase un novo par, o que limita a ventá de uso en caso de compromiso.

Os accessToken almacénanse en memoria no frontend (variables de módulo en axiosInstance.js), sen persistencia en localStorage nin cookies, o que os protexe de ataques XSS. Os refreshToken almacénanse como hash en base de datos, polo que o token real nunca queda rexistrado de forma recuperable.

O ciclo de vida da sesión no frontend xestiónase mediante un interceptor de Axios: cada petición inxecta o accessToken na cabeceira Authorization; en caso de resposta 401, o interceptor intenta un refresco silencioso e, se falla, emite un evento DOM session-expired que propaga a redirección ao login sen interacción da usuaria.

Os contrasinais almacénanse con hash bcrypt mediante BCryptPasswordEncoder de Spring Security. As comunicacións deben realizarse sobre HTTPS en produción para garantir a confidencialidade dos tokens en tránsito.

7.2. Implementación

Tecnoloxías utilizadas

O backend implementouse en Java 25 con Spring Boot 4.0.4, usando Spring MVC para a capa HTTP, Spring Security para autenticación e autorización, Spring Data JPA con Hibernate como ORM, e jjwt 0.13.0 para a xeración e validación de JWT. O envío de correos electrónicos xestiónase con JavaMailSender; en desenvolvemento úsase Mailpit como servidor SMTP local. A base de datos de desenvolvemento é H2 en memoria; a de produción, PostgreSQL con a dependencia postgres-jdbc preparada para uso xeoespacial futuro. Lombok elimina boilerplate de getters, setters e construtores.

O frontend implementouse en React 19 con Vite 8 como ferramenta de build. O enrutamento é client-side con React Router 7. As peticións HTTP usan Axios con interceptores para a xestión automática de tokens. A visualización cartográfica usa Leaflet 1.9 con tiles de OpenStreetMap. O estado global xestiónase con Zustand 5. A internacionalización usa react-i18next con ficheiros de tradución JSON en galego e inglés, cargados en runtime mediante i18next-http-backend.

Estrutura do código

O backend organízase no paquete raíz explora.map, dividido en subcapas: controller, service (interfaces e implementacións), repository, entity, dto, security e config. Toda a nomenclatura (paquetes, clases, métodos e campos) está en galego, o que mantén coherencia co idioma do proxecto.

O frontend organízase en src/pages (unha páxina por ruta), src/components (compoñentes reutilizables), src/services (funcións de chamada á API, un ficheiro por recurso), src/store (AuthContext e stores Zustand), src/hooks e src/assets/styles (CSS global por módulo). Os estilos usan clases globais BEM con variables CSS definidas en global.css; non se usan CSS Modules.

Integración de compoñentes e servizos externos

Nominatim: ao crear ou editar un mapa, `MapaServiceImpl` chama a <https://nominatim.openstreetmap.org/reverse> coas coordenadas do mapa para obter a localización en linguaxe natural (cidade, rexión, país, código de país). O parseo da resposta JSON é manual. Se a chamada falla, os campos de localización quedan a null sen interromper a operación principal.

OpenStreetMap / Leaflet: o frontend carga os tiles cartográficos directamente desde os servidores de OpenStreetMap a través de Leaflet. Non se require ningún token nin API key, en coherencia co principio de independencia de servizos propietarios do proxecto.

Mailpit: en desenvolvemento, os correos de verificación de conta envíanse a un servidor Mailpit local que os captura sen reenvialos, o que permite probar o fluxo completo de rexistro sen necesidade dun servidor SMTP real.

Descrición das principais funcionalidades implementadas

Sistema de autenticación e verificación de conta. O rexistro crea a conta nun estado non verificado e envía un correo con un token de activación de uso único e expiración de 24 horas. Ata que a conta non se verifica, o login é rexeitado. Este fluxo protexe contra a creación masiva de contas con correos falsos. Un `useRef` guard no compoñente `VerificarPage` evita que o modo estrito de React consuma o token dúas veces na montaxe do compoñente.

Sistema de mapas colaborativos con roles. Cada mapa privado ten asociada unha táboa de membros (`MapaMembro`) con rol individual por usuaria. O proceso de incorporación ocorre mediante convites: a propietaria ou unha administradora do mapa envía un convite a outra usuaria, que pode aceptalo ou rexeitalo. Ao aceptar, créase a entrada en `MapaMembro` co rol especificado no convite. Os roles determinan os permisos de forma granular: `MEMBRO` só pode ler; `COLABORADORA` pode crear elementos e editar ou eliminar os que ela mesma creou; `ADMIN_MAPA` pode editar e eliminar calquera elemento e xestionar membros. A propietaria ten sempre permisos totais independentemente da táboa de membros.

Historial de cambios. Cada operación de creación, edición ou eliminación sobre mapas, marcadores e categorías xera automaticamente unha `EntradaHistorial`. O historial é consultable con filtros por tipo de acción e por nome de usuaria. A usuaria almacénase como `String` no historial para que os rexistros persistan aínda que a conta da usuaria sexa eliminada posteriormente.

Explorador de mapas públicos. A páxina principal permite buscar mapas públicos por proximidade xeográfica. O backend filtra os mapas públicos usando a fórmula de Haversine calculada en Java, comparando as coordenadas de cada mapa co punto de busca. A busca de lugares usa Nominatim no frontend con debounce de 400 ms. Os mapas resultado pódense gardar como favoritos para acceso rápido desde a lapela "Gardados" da listaxe de mapas.

Internacionalización. A interface soporta galego e inglés. O idioma detéctase na orde: preferencia gardada en localStorage, e como último recurso, o idioma do navegador. A preferencia sincronízase co perfil da usuaria no backend para persistir entre dispositivos. O cambio de idioma é instantáneo sen recargar a páxina.

Eliminación de conta. A eliminación execútase nunha única transacción que limpa en orde os tokens de sesión, os gardados, as membresías en mapas alleos, os convites (como anfitriño e como convidada), os mapas propios con todo o seu contido en cascada, e finalmente a entidade de usuaria. O historial de accións da usuaria en mapas alleos pervive, identificado co nome de usuaria como texto, para non comprometer a trazabilidade de eses mapas.

7.3. Probas e validación

Estratexia de probas

A estratexia de probas do proxecto combina dous niveis complementarios: probas unitarias illadas sobre compoñentes de lóxica pura, e probas de integración que exercitan o sistema de punta a punta desde a capa HTTP ata a base de datos.

As probas unitarias aplícanse sobre clases e funcións con lóxica suficientemente complexa e illable, onde a verificación sen dependencias externas aporta valor real. As probas de integración cobren os fluxos completos do sistema, en particular a lóxica de control de acceso por rol, que é o aspecto máis crítico desde o punto de vista da seguridade e o máis difícil de verificar manualmente de forma sistemática.

A selección dos casos de test priorizou a cobertura dos escenarios de maior risco: autenticación, validación de tokens, control de acceso a recursos e diferenciación de comportamento entre roles.

Probas unitarias

Implementáronse tres suites de tests unitarios puros, sen levantar o contexto de Spring nin montar compoñentes React.

JwtUtilsTest.java — Xeración e validación de tokens JWT (3 tests)

Verifica de forma illada a lóxica de JwtUtils, instanciando a clase directamente con new JwtUtils() e inxectando os valores de configuración mediante ReflectionTestUtils.setField():

Test	Escenario	Resultado esperado
T1	Token xerado é válido e contén o username correcto	validarToken → true, obterUsernameDeToken → username correcto
T2	Token con expiración de 1ms é rexeitado tras agarda	validarToken → false
T3	Token con firma manipulada é rexeitado	validarToken → false

MapaAccesoServiceTest.java — Lóxica de control de acceso a mapas (4 tests)

Verifica de forma illada a lóxica de MapaAccesoService, mockeando MapaRepository e MapaMembroRepository con Mockito mediante @InjectMocks e @Mock, sen acceso á base de datos:

Test	Escenario	Resultado esperado
T1	Mapa público accedido por calquera usuaria	Sen excepción
T2	Mapa privado accedido pola propietaria	Sen excepción
T3	Mapa privado accedido por membro con entrada en MapaMembro	Sen excepción
T4	Mapa privado accedido por usuaria foránea sen entrada	IllegalStateException

usePasswordStrength.test.js — Cálculo de forza de contrasinal (3 tests)

Verifica de forma illada a función calcularForza exportada desde o hook usePasswordStrength.js, sen montar ningún compoñente React, usando Vitest:

Test	Escenario	Resultado esperado
T1	Contrasinal curto só con minúsculas ("abc")	forza == 'débil', percentaxe ≤ 33
T2	Contrasinal con maiúsculas e números ("Abc123")	forza == 'moderado', percentaxe entre 34 e 66
T3	Contrasinal con maiúsculas, números e símbolos ("Abc123!\$")	forza == 'forte', percentaxe > 66

Probas de integración

Implementouse unha suite de 15 tests de integración no ficheiro `PermisosIntegrationTest.java`, usando `@SpringBootTest` con `MockMvc` e base de datos H2 en memoria. Cada test é independente grazas a `@Transactional` con rollback automático ao remate.

Os tests organízanse en tres grupos:

Grupo A — Autenticación e validación de JWT (4 tests)

Verifican que a capa de seguridade funciona correctamente de forma independente da lóxica de negocio:

Test	Escenario	Resultado esperado
T6	Request sen cabeceira JWT	401
T7	JWT con firma manipulada	401
T8	Login con contrasinal incorrecto	401
T9	Login con conta non verificada	403

Grupo B — Control de acceso por rol (5 tests)

Verifican que o sistema de permisos por rol (MEMBRO, COLABORADORA, ADMIN_MAPA, propietaria) se aplica correctamente sobre as operacións de escritura:

Test	Escenario	Resultado esperado
T1	Usuaria foránea accede a mapa privado	403
T2	MEMBRO intenta crear marcador	403
T3	COLABORADORA crea marcador propio	201
T4	COLABORADORA elimina marcador alleo	403
T5	Propietaria elimina calquera marcador	204

Grupo C — Funcionalidade e casos límite (6 tests)

Verifican operacións CRUD básicas e escenarios de interacción entre roles:

Test	Escenario	Resultado esperado
T10	Propietaria crea categoría	201
T11	Propietaria lista marcadores do seu mapa	200 + array non baleiro
T12	Propietaria edita o seu mapa	200
T13	Eliminación de mapa elimina contido en cascada	204 + 404 no mapa eliminado
T14	ADMIN_MAPA edita marcador alleo	200
T15	Fluxo completo de convite: bloqueo previo, aceptación e acceso posterior	403 → 200 → 200

A execución completa da suite (mvn test) produce o seguinte resultado:

Tests run: 15, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Probos de usuaria

As probas de usuaria realizáronse de forma manual sobre a interface web con ambos servidores en execución (backend en localhost:8080, frontend en localhost:5173). Verificáronse os fluxos principais da aplicación seguindo os casos de uso definidos na sección 2.5:

- Rexistro, verificación de correo e inicio de sesión.
- Creación, edición e eliminación de mapas.
- Engadir, editar e eliminar marcadores e categorías.
- Filtrado de marcadores por categoría.
- Envío e aceptación de convites entre usuarias.
- Visualización do historial de cambios.
- Cambio de rol de membros por parte dunha administradora do mapa.
- Navegación entre as tres lapelas da lista de mapas (proprios, colaboracións, gardados).

Resultados e incidencias detectadas

A execución completa de todas as suites produce os seguintes resultados:

Backend (mvn test):

Tests run: 22, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Frontend (npm test):

Test Files 1 passed

Tests 3 passed

As suites pasaron sen fallos desde a primeira execución completa, o que indica que tanto a lóxica de control de acceso como a xeración e validación de tokens e o cálculo de forza de contrasinal están correctamente implementados.

Durante as probas manuais de usuaria detectáronse as seguintes incidencias, corrixiadas durante o proceso de desenvolvemento:

- **Dobre consumo do token de verificación en React StrictMode:** o efecto de verificación executábase dúas veces en modo desenvolvemento, consumindo o token na primeira chamada e fallando na segunda. Resolto mediante un `guard useRef` que impide a segunda invocación.
- **Pechado incorrecto de menús despregables en contedores con overflow: hidden:** os menús kebab das tarxetas de mapa quedaban recortados polo contedor pai. Resolto mediante `ReactDOM.createPortal` con coordenadas absolutas obtidas de `getBoundingClientRect`.
- **Desincronización do estado de categorías tras eliminar un marcador:** ao eliminar un marcador cunha categoría asignada, a lista de categorías non se actualizaba visualmente. Resolto elevando o estado de categorías a `MapaDetallePage` e coordinando as recargas de marcadores e categorías no mesmo callback.

7.4. Despregamento e documentación

Configuración do entorno de execución

Instrucións de instalación e despregamento

Manual de usuaria

Manual técnico

7.5. Resultados e avaliación

Cumprimento dos obxectivos

Valoración do rendemento e usabilidade

Comparativa con solucións existentes

7.6. Conclusións e liñas de mellora

Conclusións xerais

Limitacións do proxecto

Posibles melloras e traballos futuros

Anexos

1. Diagrama da arquitectura do backend
2. Diagrama da arquitectura do frontend
3. Modelo Entidade-Relación Conceptual de Chen
4. Diagrama UML de clases
5. Mockups
6. Guía de estilo

Referencias